

Welcome back. You are signed into your member account
bg....@jaxondigital.com. [Not you?](#)



Me



AI Advances · [Follow publication](#)

★ Member-only story

FinOps for AI Agents: when Eval is more Expensive than Dev

the hidden cost of non-functional LLM calls for Agentic AI

15 min read · 4 days ago



Debmalya Biswas

Follow



Listen



Share



More

1. Introduction

The discussion around ChatGPT (in general, generative AI), has now evolved into agentic AI. While ChatGPT is primarily a chatbot that can generate text responses, AI agents can execute complex tasks autonomously, e.g., make a sale, plan a trip, make a flight booking, book a contractor to do a house job, order a pizza. Fig. 1 below illustrates the evolution of agentic AI systems.

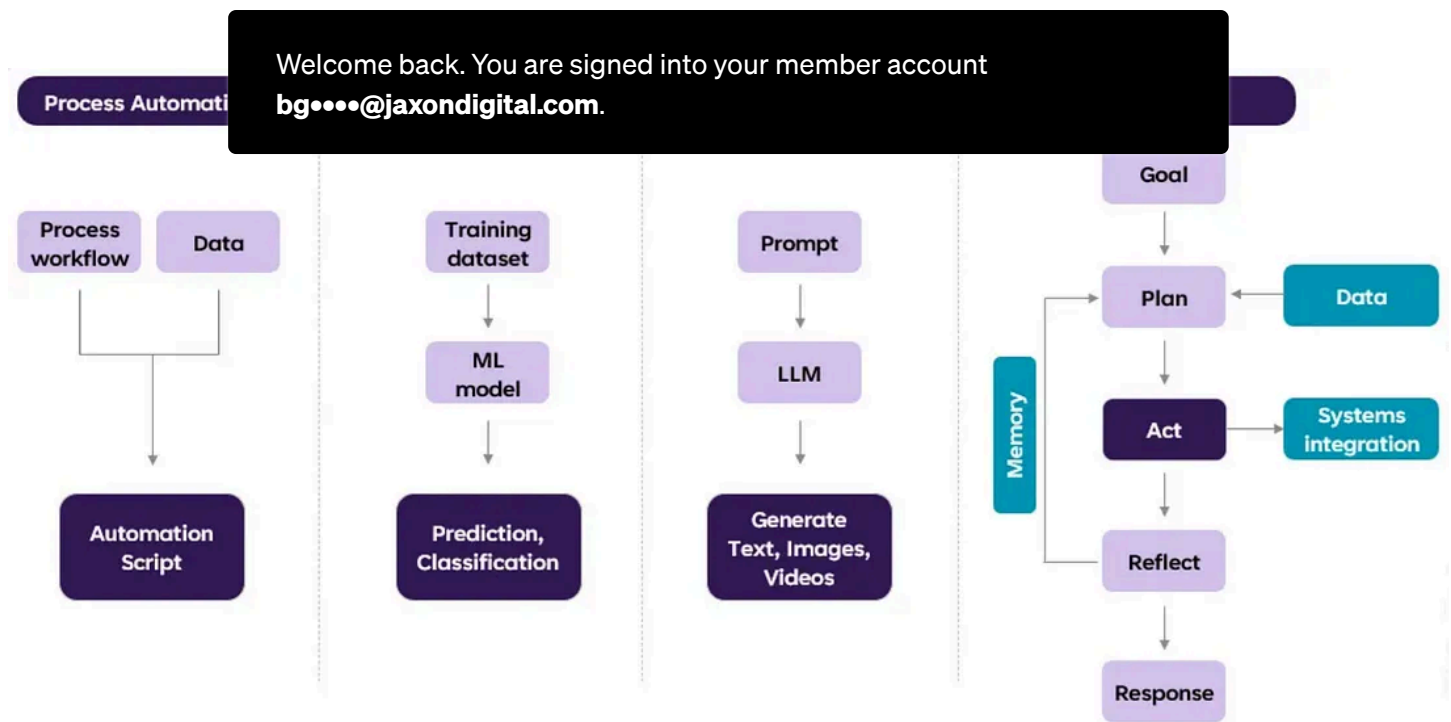


Fig. 1: Agentic AI evolution (Image by Author)

Bill Gates recently envisioned a future where we would have an AI agent that is able to process and respond to natural language and accomplish a number of different tasks. Gates used planning a trip as an example.

Ordinarily, this would involve booking your hotel, flights, restaurants, etc. on your own. But an AI agent would be able to use its knowledge of your preferences to book and purchase those things on your behalf.

The key characteristics of agentic AI systems are their autonomy and reasoning prowess that allow them to decompose complex tasks into smaller executable tasks, and then orchestrate their execution with integrated (external) tools in a way that can monitor, reflect, and self-correct the execution as and when needed. Given this,

agentic AI has the potential to disrupt almost every business process prevalent in an enterprise today.

As agentic AI adoption accelerates in the enterprise, the focus is shifting from agent development to deploying them in a **cost efficient** and governed manner. Yes, agents

can execute workflow processes efficiently, but that comes at a cost. As such, cost is becoming a critical factor in the adoption of agentic AI for investment brokering,

Welcome back. You are signed into your member account
bg****@jaxondigital.com.

we do anticipate agentic brokering platforms in the near-future where agentic providers would be able to bid for a task — with the execution awarded to the most reliable and cost-efficient bidder.

Given this, we focus on the cost aspect of agents in this article, esp. the cost of **LLM invocations** for an agentic workflow. We all know LLM invocations are expensive, so to achieve FinOps excellence with respect to LLM invocations; we need to understand the purpose underlying LLM invocations during an agentic workflow execution and how to optimize them.

When we think about LLM invocations for an agentic execution today, we primarily think of LLMs for the

- reasoning step: given a goal and registry of available agents, generate the optimal plan / orchestration graph of agents to execute that plan.
- agent execution: where an LLM call might be needed to fulfill the agent functionality, e.g., summarize, generate a personalized email, etc.

However, we highlight in this article that

non-functional aspects, e.g., memory, evaluation and guardrails; can lead to 2–3 times more LLM invocations — than those invoked to directly execute the agent functionality.

The rest of the article is organized as follows. We first outline the reference architecture of an agentic AI platform in section 2. In section 3, we outline what FinOps means for agentic AI, taking a holistic look at all factors involved, such as, model, compute, storage, network. We deep-dive into the “model” cost aspect in section 4, explaining what LLM inferencing means in an agentic context, with a specific focus on non-functional LLM invocations. Section 5 concludes the article with some directions for future work.

2. Agentic AI Reference Architecture

In this section, we outline the key modules of a reference agentic AI platform — illustrated in Fig. 2:

- **Reasoning module:** to decompose complex tasks and adapt their execution to achieve
- **Agentic marketplace:** of existing and available agents;
- **Orchestration module:** to orchestrate and monitor (observe) the execution of multi-agent systems;
- **Integration module:** MCP tools to integrate with enterprise systems, e.g., ERP, CRM, KB repositories;
- Shared **memory** management for data and context sharing among agents;
- **Governance** layer, including explainability, privacy, security, safety guardrails, etc.

Fig. 2: Agentic AI platform reference architecture (Image by Author)

Given a user task, the goal of the agentic AI platform is to identify (compose) an agent (group of agents) capable to executing the given task. So the first component we need is a **reasoning** module capable of decomposing a task into sub-tasks, with execution of the respective agents orchestrated by an orchestration engine.

Chain of Thought (CoT) is the most widely used decomposition framework today to transform complex tasks into multiple manageable tasks and shed light into an

interpretation of the model's thinking process. Further, the ReAct (reasoning and acting) framework enables agents to iteratively refine their reasoning and outputs, learn from past interactions, and adapt to new information.

Welcome back. You are signed into your member account
bg****@jaxondigital.com.

Agent composition implies the existence of an agent marketplace / registry of agents — with a well-defined description of the agent capabilities and constraints. For example, the Agent2Agent (A2A) protocol specifies the notion of an Agent Card (a JSON document) that serves as a digital “business card” for agents. It includes the following key information:

```
Identity: name, description, provider information.  
Service Endpoint: The url where the A2A service can be reached.  
A2A Capabilities: Supported protocol features like streaming or pushNotifications.  
Authentication: Required authentication schemes (e.g., "Bearer", "OAuth2") to interact.  
Skills: A list of specific tasks or functions the agent can perform (AgentSkillSet).
```

Given the need to orchestrate multiple agents, there is a need for a **system integration module** supporting different agent interaction patterns, e.g., agent-to-agent API, agent API providing output for human consumption, human triggering an AI agent, AI agent-to-agent with human in the Loop. The integration patterns need to be supported by the underlying Agent OS platform.

It is also important to mention that integration with enterprise systems (e.g., CRM in this case) will be needed for most use-cases. Refer to the Model Context Protocol (MCP) from Anthropic that seems to have become the de-facto standard to connect AI agents to external systems (where enterprise data resides).

Given the long-running nature of complex agents, memory management is key for agentic AI systems.

This entails both context sharing between tasks and maintaining execution context over long periods.

The standard approach here is to save the embedding representation of agent information into a vector store database that can support maximum inner product search (MIPS). For fast retrieval, the approximate nearest neighbors (ANN)

algorithm is used that returns approximately top k-nearest neighbors with an accuracy of 95%.

Welcome back. You are signed into your member account
bg...@jaxondigital.com.

Finally, the governance module. we need to ensure that data shared by the user is specific to a task, or user profile data that cuts across tasks; is only shared with the relevant agents (table / report authentication and access control). Refer to my previous [article](#) on Responsible AI Agents for a discussion on the key dimensions needed to enable a well governed AI agent platform in terms of hallucination guardrails, data quality, privacy, reproducibility, explainability, human-in-the-loop (HITL), etc.

3. Agentic AI Cost Considerations

FinOps for agentic AI can be defined as a

best practice to bring together finance, engineering, and business — to manage Agentic AI costs by maximizing value and ensuring financial accountability.

It involves using data-driven insights to manage trade-offs between agility, governance, cost vs. RoI, empowering enterprises to optimize AI spend proactively through resource right-sizing and efficient allocation.

In a typical agentic AI scenario, it would be a combination of:

- compute infrastructure
- model: large language models (LLMs) / small language models (SLMs)
- storage: memory, vector databases for search, etc.

Let us consider a reference scenario: LangGraph as the agentic development framework with deployment on Azure Kubernetes Service (AKS):

- [LangGraph](#) orchestrates agent execution (via internal APIs or managed runtime).
- Agent is deployed with custom logic or tools — as a containerized agent endpoint on AKS.
- [AI Search](#) indexes enterprise data (vector + text) and acts as the retrieval-augmented generation (RAG) knowledge source.
- AKS pods call AI Search and/or model endpoints (say Azure OpenAI GPT or fine-tuned LLMs / SLMs in Azure Foundry).

- OpenTelemetry logs stored in Azure Monitor with Application Insights.

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

The cost can be estimated based on AKS pod read / write, search query latency:

- How many agent pods run concurrently? The average agent container image size would be 2–4 GB per agent × concurrent sessions.
- How many LLMs / checkpoints are staged in AKS or cached? The average here would be 1–10 GB (tokenizer, local weights, embeddings cache).
- Traffic between AKS ↔ AI Search / AI Foundry and latency say < 200 ms.
- Vector storage in terms of the size of embeddings (indexed in AI Search) can vary between 100 MB to 100 GB depending on the use-case.
- Log volume around 100 MB per day per 100 sessions.

For 5 agents running concurrently, the representative volumetrics would be:

- 5 pods × 2 vCPU × 6 GB RAM each → 10 vCPU, 30 GB RAM
- Cache 5 GB per pod → 25 GB ephemeral SSD
- Log 1 GB per day → 10 GB per day telemetry
- Vector data ~25 GB total, stored in AI Search

While the above focused on understanding the **agentic infrastructure cost**, we focus on analysing the LLM invocation calls in the next section — given that they still make up a bulk of the overall agentic system cost.

4. LLM Invocation Cost

In this section, we deep-dive into LLM inferencing aspects, e.g., observability, latency, throughput, non-determinism, etc. — critical to deploying multi-agent systems (MAS) at scale.

We first consider the dimensions impacting LLM inferencing, e.g.,

- input and output context window
- model size,

- first-token latency, inter-token latency, last token latency;

- through

Welcome back. You are signed into your member account
bg●●●@jaxondigital.com.

We then extrapolate the same to agentic AI:

- mapping token **latency** to latency of executing the first agent vs. the full agentic orchestration
- considering the output of (preceding) agent together with the overall execution state / contextual understanding as part of the input **context window** size of the following agent; and finally
- accommodating the inherent non-determinism in agentic executions. In particular, we introduce the notion of **compensation** as a rollback strategy to accommodate agentic goal changes and execution failures.

4.1 LLM Inference Sizing

LLM inference sizing depends on many use-case dimensions, e.g.,

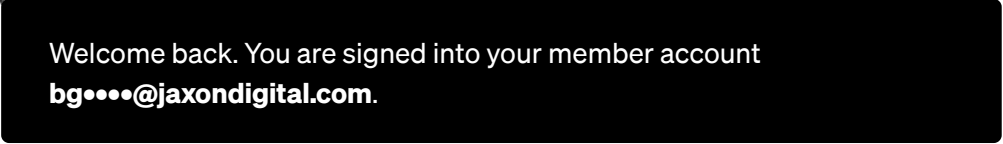
- input and output context window: high-level, words are converted into tokens, and models like Llama run on about 4k-8k tokens or roughly 3000–6000 words in English.
- model size: are we running the model at full precision, or a quantized version?
- first-token latency, inter-token latency, last token latency; and, finally
- throughput: defined as the number of requests an LLM can process in a given period.

Let us consider the **batch** scenario first. Here, we mostly know our input and output context lengths; so the focus is on optimizing throughput. (Latency is not relevant here given the offline / batch nature of the execution.) To achieve high throughput:

- Determine if your LLM fits in one GPU?
- If not, apply pipeline / tensor parallelism to optimize the number of GPUs needed. Then, just increase the batch size to be as large as possible.

For the **streaming** scenario, we need to consider the trade-off between throughput and latency. To understand latency, let us take a look at the processing stages of a

typical LLM request: Prefill and Decoding (illustrated in Fig. 3)

A screenshot of a chat interface. It shows a single message in a dark-themed window. The message text is "Welcome back. You are signed into your member account" followed by a redacted email address "bg...@jaxondigital.com". The text is white on a black background.

Welcome back. You are signed into your member account
bg...@jaxondigital.com.

Fig. 3: LLM processing stages: Prefill & Decoding (Image by Author)

Prefill is the latency between pressing ‘enter’ and the first output token appearing on the screen. Decoding occurs when the other words in the response are generated. In most requests prefill takes less than 20% of the end-to-end latency, while decoding takes more than 80%.

Given this, most LLM implementations tend to send tokens back to the client as soon as they are generated — to reduce latency.

To summarize, in streaming mode, we primarily care about the time to first token, as this is the time during which the client is waiting for the first token. Afterwards, the following tokens are generated much faster, and the rate of generation is usually faster than the average human reading speed.

Note that for RAG pipelines, even the first-token latency can be significantly high.

RAGs typically target the full context window as a result of adding chunks of documents to the input prompt. In sequential model, we have to wait for the end result; and as a result we care about the end-to-end latency. This is the time to produce all the tokens in the (response) output sequence.

Finally, reg. the trade-off between latency & throughput:

increasing the batch size (running multiple requests through the LLM concurrently) tends to make latency worse but throughput better.

Of course, upgrading the underlying hardware / GPU can improve both throughput and latency. Refer to Nvidia’s tutorial on [LLM inference sizing](#) for a detailed discussion on this topic.

4.2 Agentic AI Inference Sizing

In this section, we highlight the key steps in extrapolating LLM to agentic AI inferencing.

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

Fig. 4: Agentic AI inferencing dimensions (Image by Author)

Latency

Token latency maps to agent processing latency. The first-token versus end-to-end token latency discussion maps to first-agent versus end-to-end execution latency of the full orchestration / decomposed plan in this case.

We thus need to balance the requirement of streaming agent execution outputs as soon as they finish their execution versus outputting the result once execution of the full orchestration has terminated.

For a detailed discussion, refer to my previous [article](#) on stateful representation of AI agents enabling both real-time and batch observability of the agentic orchestration.

Context Window Size

The output of one agent becomes the input of the next agent to be executed in a multi-agent orchestration. So it is very likely that (at least some part of) the preceding agent output together with the overall execution state / contextual understanding (stored in the memory management layer) will become part of the

input context passed to the following agent — and this needs to be taken into account as

Welcome back. You are signed into your member account
bg●●●●@jaxondigital.com.

4.3 LLM Inv

We have so far built an understanding of the factors impacting LLM invocation costs, in an agentic context. This needs to be multiplied by the number of times an LLM gets invoked during an agentic execution lifecycle — illustrated in Fig. 5.

Fig. 5: LLM invocations during an Agentic lifecycle (Image by Author)

So it is important to understand the different scenarios in which an LLM can get invoked during an agentic execution lifecycle.

While most of the focus is on reasoning based LLM invocation calls to generate the execution plan (given a goal), and to execute the corresponding agents' functionality; we show that

an equal or more number of LLM invocations need to be made for non-functional aspects, e.g., memory, evaluation, and guardrails.

Functional LLM invocations

Given a user task, we prompt an LLM for the task decomposition — refer to the agentic platform architecture illustrated in Fig. 2. Unfortunately, this also means that agentic AI systems today are limited by the **reasoning** capabilities of LLMs. For ex., the GPT4 task decomposition of the prompt

Generate a list of alternative products to increase sales of US\$100K. Product details and their performance metrics are available at [url]. Connect to CRM system [integration] for customer names, email addresses, and demographic details.

Welcome back. You are signed into your member account
bg●●●●@jaxondigital.com.

is detailed in Fig. 6: (Analyze products) — (Identify target audience) — (Create tailored email campaign).

Fig. 6: LLM invocations during the Reasoning and Reflect / Adapt stages (Image by Author)

The LLM then monitors the execution / environment and adapts autonomously as needed. In this case, the agent realised that it is not going to achieve its sales goal and autonomously added the tasks:

(Find alternative products) — (Utilize customer data to personalize the emails) — (Perform A/B testing).

Non-functional LLM invocations — Memory Management

Here, we briefly expand the memory router functionality to highlight the LLM invocations involved in agentic memory management — illustrated in Fig. 7 (green boxes). For a detailed discussion of the topic, refer to my previous article on [Long-term Memory for AI Agents](#).

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

Fig. 7: LLM invocations in Agentic memory management (Image by Author)

The memory router, always, by default, routes to the long-term memory (LTM) module to see if an existing pattern is there to respond to the given user prompt. If yes, it retrieves and immediately responds, personalizing it as needed.

If the LTM fails, the memory router routes it to the short-term memory (STM) module which then uses its retrieval processes (function calling, APIs, etc.) to get the relevant context into the STM (working memory) — leveraging applicable data services.

The STM — LTM transformer module (implemented using LLM invocations) is always active and constantly getting the context retrieved and extracting recipes out of it (e.g., refer to the concepts of teachable agents and recipes in [AutoGen](#)) and storing in a semantic layer (implemented via Vector DB). At the same time, it is also collecting other associated properties (e.g., number of tokens, cost of executing the response, state of the system, tasks executed / responses generated) and

- creating an episode which is then getting stored in a knowledge graph
- with the underlying procedure stored in a finite state machine (FSM).

Non-functional LLM invocations — Evaluation Layer

Defining a comprehensive agentic evaluation strategy is a multi-faceted problem with the need to balance functional and non-functional requirements.

Welcome back. You are signed into your member account
bg****@jaxondigital.com.

- the underlying LLM (reasoning model),
- solution architecture (RAG, fine-tuning, agent / tool orchestration pattern, etc.),
- applicable enterprise policies and AI regulations / responsible AI guidelines.

Fig. 8: Agentic AI evaluation strategy (Image by Author)

There are primarily 3 types of evaluation methodologies prevalent today:

- Generic benchmarks and datasets
- LLM-as-a-Judge
- Manual evaluation

The LLM-as-a-Judge method uses an “evaluation” LLM (another pre-trained LLM) to evaluate the quality of responses of the target LLM, scoring them using methods like LangChain’s CriteriaEvalChain. Unfortunately, the use-case specific limitations persist in this case as well.

It has the advantage of accelerating the LLM evaluation process, though (in most cases) at a higher cost given the use of a second LLM.

Non-functional LLM invocations — Agentic Guardrails

Welcome back. You are signed into your member account

bg....@jaxondigital.com.

With more we are seeing increasing focus on their ~~risks~~. Rather than creating a new list, I tried to consolidate the risks identified in the below two references: (For a detailed discussion of the topic, refer to my previous article on [Guardrails for AI Agents](#))

1. OWASP [whitepaper](#): Agentic AI — Threats and Mitigations, 2025.
2. IBM [whitepaper](#): Accountability and Risk Matter in Agentic AI, 2025.

R1–15 refer to the risks identified in [1]. The ones in brackets () refer to the corresponding risks identified in [2]. R16: Persona-driven **Bias**, e.g., is quite interesting, which has been identified in [2], but is missing from [1].

- R1: Misaligned & Deceptive Behaviors (Dynamic Deception)
- R2: Intent Breaking & Goal Manipulation (Goal Misalignment)
- R3: Tool Misuse (Tool/ API Misuse)
- R4: Memory Poisoning (Agent Persistence)
- R5: Cascading Hallucination Attacks (Cascading System Attacks)

(Security Vulnerabilities)

- R6: Privilege Compromise
- R7: Identity Spoofing & Impersonation
- R8: Unexpected RCE & Code Attacks

(Operational Resilience)

- R9: Resource Overload
- R10: Repudiation & Untraceability

(Multi-agent Collusion)

- R11: Rogue Agents in Multi-agent Systems
- R12: Agent Communication Poisoning

- R13: Human Attacks on Multi-agent Systems

(Human O

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

- R14: Human Manipulation
- R15: Overwhelming Human in the Loop
- R16: (Persona-driven Bias)

The interesting part from a risk mitigation point of view is that their mitigation is often left to a central **guardrails** layer. However, this is not realistic and

the guardrails need to be specific to the underlying use-case, and implemented in their respective platform components / layers — which has a direct impact on the overall solution architecture.

The agentic AI component risk-architecture mapping is depicted in Fig. 9. As can be imagined, each guardrail implementation maps to one or more LLM/SLM invocations.

Fig. 9: Agentic AI guardrails mapping to Platform architecture (Image by Author)

5. Conclusion

While the benefits of agentic AI systems are evident, they are also complex systems that are difficult to manage.

Welcome back. You are signed into your member account
bg****@jaxondigital.com.

Towards this end, we outlined the key cost dimensions impacting agentic AI systems to achieve FinOps excellence. We started by identifying the key architectural components in an agentic platform, mapping them to first analyzing the infrastructure cost — to then deep-diving into the LLM invocation costs.

We highlighted that LLM invocation calls for non-functional aspects, e.g., memory, evaluation, guardrails, etc. can actually outnumber the agentic functionality / reasoning calls — and hence need to be factored in as a first class citizen for costing exercises.

We believe that both aspects are critical to moving agentic AI solutions in production, and that this work will contribute significantly towards driving their enterprise adoption.

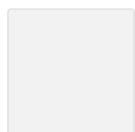
Agentic Ai

Finops

Large Language Models

Generative Ai Tools

Agentic Ai Framework



Follow

Published in AI Advances

53K followers · Last published 5 hours ago

Democratizing access to artificial intelligence



Follow

Written by Debmalya Biswas

4.7K followers · 176 following

Welcome back. You are signed into your member account
bg**@jaxondigital.com.**

Responses (7)



Bgerby

What are your thoughts?



Jukka Paulin

4 days ago



integration patterns need to be supported by the underlying Agent OS platform.

this is a really good point! Perhaps also for the fact that some collection of agents need to be ensured for the fact that they will be used for only the intended purposes, ie. to ensure they're not going to be part of a "rogue pack".



6



1 reply

[Reply](#)



Ricky Wong

20 hours ago (edited)



This article is spot on ... running agents in production (vs POC) could cost a lot more than most people think. In fact, it is harder to estimate the cost than traditional IT systems because too many steps are non-deterministic, e.g. how many turns... [more](#)



10



1 reply

[Reply](#)



Gulesehar khan

23 hours ago



The irony of Agentic AI is clear: evaluation can quietly drain more resources than development itself.



5



1 reply

Reply

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

See all responses

More from Debmalya Biswas and AI Advances



In Data Science Collective by Debmalya Biswas

Guardrails for AI Agents

Use-case specific validation tests and guardrails generation for Agentic AI



Aug 4



433



11



Welcome back. You are signed into your member account
bg....@jaxondigital.com.

 In AI Advances by Nikhil Anand

I wasted months running slow LLMs before learning this

Why your LLM is running at just 10% of its potential speed

★ Oct 10 🖱 934 💬 11



 In AI Advances by Nikhil Anand

If you still think AI is magic, read this.

AI might seem mysterious and weird, but it's a lot simpler than you think.

★ Oct 14 🖱 576 💬 15



Welcome back. You are signed into your member account
bg**@jaxondigital.com.**



In Data Science Collective by Debmalya Biswas

Agentic AI MCP Tools Governance

Discovery and governance guidelines for AI Agents in the Enterprise

★ Sep 16 🖱️ 331 💬 7



See all from Debmalya Biswas

See all from AI Advances

Recommended from Medium

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

 Dr Adrian Lucas Malec

How I Built Lightning-Fast Vector Search for Legal Documents

From 53 to 2,880 queries/sec with USearch

Oct 19  395  5



 In AIGuys by Vishal Rajput 

Kimi K2 Just Killed OpenAI, & Claude

If one thing tech bros are better than at building AI, it is creating hype around AI. And the release of Kimi K2 just proved the opposite...

★ Nov 8



Welcome back. You are signed into your member account
bg....@jaxondigital.com.

 Aruna Pattam

Agentic AI: Part 8—The Agentic AI Reference Architecture

Why Architecture Matters

Nov 6  136  2



 In AI Software Engineer by Joe Njenga

Anthropic Just Solved AI Agent Bloat—150K Tokens Down to 2K (Code Execution With MCP)

Anthropic just announced that their new AI model, Claude 3, is faster by 98%,
shift from to

Welcome back. You are signed into your member account
bg....@jaxondigital.com.

★ Nov 6



 In Coding Nexus by Code Coup

Google DS-STAR: The AI Agent That Automates Data Science from Analysis to Visualization

If you've ever juggled five different file types while trying to answer a simple data question, you'll appreciate what Google has just...

★ 6d ago  50  1



Welcome back. You are signed into your member account
bg....@jaxondigital.com.

 In Medialesson by Marius Schröder

JSON vs TOON—A new era of structured input?

Why structure matters more than ever

Nov 3  797  42



See more recommendations